

# KOBE-Tech README

神戸電子専門学校 ソフト4 チーム制作（1回目）で作成した Web アプリケーションです。 学生・教員が記事／質問／回答／制作物を投稿し、知識共有やフィードバックを行えるナレッジ共有サービスです。

## 目次

- 1. 実行環境
- 2. インストール方法（開発・実行に必要なツールの導入）
- 3. 起動方法（git clone からローカルで動かすまで）
- 4. デプロイ環境（公開URL）
- 5. 簡単な操作方法
- 6. 評価してほしいポイント
- 7. 苦労した点・工夫した点

## 1. 実行環境

本アプリは Web アプリケーションのため、PC へインストールして使うソフトではありません。ブラウザからアクセスして利用します。 ローカルで動かす場合は、以下の環境で開発・動作を確認しています。

### 構成

| 領域          | 技術                                          |
|-------------|---------------------------------------------|
| フロントエンド     | Vue 3 / Vite / TypeScript / Vuetify / Pinia |
| バックエンド      | Go 1.25 / Gin                               |
| データベース      | PostgreSQL 16                               |
| オブジェクトストレージ | MinIO（画像などのファイル保存）                          |
| API ドキュメント  | Swagger / OpenAPI                           |
| 開発環境        | Docker Compose / mise（バージョン管理）              |

### 採用バージョン

| ツール        | バージョン    | 備考                |
|------------|----------|-------------------|
| Go         | 1.25.0   |                   |
| Node.js    | v24.15.0 | LTS を採用           |
| PostgreSQL | 16       | Docker イメージで起動    |
| Docker     | （任意の最新版） | Docker Desktop 推奨 |
| air        | （最新版）    | Go のホットリロード用      |
| swag       | v1.16.4  | OpenAPI 生成用（任意）   |

## 使用ポート

| ポート  | 用途                   |
|------|----------------------|
| 5173 | フロントエンド（Vite 開発サーバー） |
| 8080 | バックエンド API サーバー      |
| 5432 | PostgreSQL           |
| 9000 | MinIO（API エンドポイント）   |
| 9001 | MinIO（管理コンソール）       |

## 2. インストール方法（開発・実行に必要なツールの導入）

Web アプリのため、アプリ本体のインストール作業はありません。ローカルで動かすために、以下のツールを事前に導入してください。

導入済みであれば、各コマンドでバージョンが表示されることを確認してください。

### 2-1. Go（バックエンド）

公式: <https://go.dev/dl/>

```
$ go version
go version go1.25.0 ...
```

### 2-2. Node.js（フロントエンド）

公式: <https://nodejs.org/>

```
$ node --version
v24.15.0

$ npm --version
```

バージョン管理ツール（mise / nvm / asdf 等）の利用を推奨します。推奨は mise です。

### 2-3. Docker / Docker Compose（DB・ストレージ）

Docker Desktop を入れると Docker / Docker Compose がまとめて揃います。公式:

<https://www.docker.com/products/docker-desktop/>

```
$ docker -v
$ docker compose version
```

## 2-4. air (Go ホットリロード)

```
$ go install github.com/air-verse/air@latest
$ air -v
```

## 2-5. swag (OpenAPI 生成 / 任意)

API 定義を再生成する場合のみ必要です。動作させるだけなら不要です。

```
$ go install github.com/swaggo/swag/cmd/swag@v1.16.4
$ swag --version
```

---

## 3. 起動方法 (git clone からローカルで動かすまで)

バックエンド (API + DB + ストレージ) とフロントエンドを、それぞれ別のターミナルで起動します。

### 3-0. リポジトリの取得

```
$ git clone https://github.com/Baymax6s/KOBE-Tech.git
$ cd KOBE-Tech
```

### 3-1. バックエンドの起動

#### 1. api ディレクトリへ移動

```
$ cd api
```

#### 2. 環境変数ファイルを作成 (.env.example から .env を生成)

```
$ make setup
```

#### 3. 起動

```
$ make dev
```

`make dev` は次の処理をまとめて行います。

- Docker Compose で PostgreSQL / MinIO を起動
- マイグレーション (テーブル作成) を自動実行

- seed データ（初期ユーザー・サンプル記事など）を投入
- air で API サーバーを起動（ホットリロード）

API サーバー: <http://localhost:8080>

DB だけ起動／停止したい場合

```
$ make db-up  
$ make db-down
```

## 3-2. フロントエンドの起動（別ターミナル）

### 1. app ディレクトリへ移動

```
$ cd app
```

### 2. 環境変数ファイルを作成

```
$ cp .env.example .env
```

### 3. 依存パッケージのインストール

```
$ npm install
```

### 4. 起動

```
$ npm run dev
```

フロントエンド: <http://localhost:5173>

## 3-3. 動作確認

ブラウザで <http://localhost:5173> を開きます。ログインを試す場合は、seed データの初期ユーザーを利用できます。

| ユーザー名 | パスワード    |
|-------|----------|
| admin | Password |
| 田中太郎  | Password |
| 山田花子  | Password |

## ユーザー名 パスワード

佐藤次郎 Password

### 参考: 管理用コンソール

- API ドキュメント (Swagger): <http://localhost:8080/swagger/>
- MinIO 管理コンソール: <http://localhost:9001> (Username: **minio** / Password: **password**)

## 4. デプロイ環境（公開URL）

公開URL: <https://vue-cjne.onrender.com/>

無料ホスティング（Render）を利用しているため、しばらくアクセスがないとサーバーがスリープ状態になります。初回アクセス時は起動に時間がかかったり、一時的に表示が不安定になることがあります。その場合は少し待ってから再読み込みしてください。基本的に自分のpcにcloneしてローカルで動かすこと推奨です。

## 5. 簡単な操作方法

### 5-1. 記事一覧を見る

アプリを開くと、まず「記事一覧」画面が表示されます。投稿された記事がカード形式で新しい順に並びます。上部の「タグで絞り込み」から、興味のあるタグで記事を絞り込むこともできます。



### 5-2. 記事の詳細を見る

記事のカードをクリックすると、記事の詳細が表示されます。本文は Markdown で表示され、コードブロックなども見やすく整形されます。

 ログアウト

## Dockerで開発環境を統一する

Docker

開発環境

 田中太郎  
投稿日 2026/04/20

### Dockerで開発環境を統一する

`docker compose` を使ってチームの開発環境を統一する Tips です。

#### 最小構成の docker-compose.yml

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: password
      POSTGRES_DB: app
    ports:
      - "5432:5432"
    volumes:
      - db-data:/var/lib/postgresql/data
```

### 5-3. 記事に「いいね」をする

記事の下部にある「いいね」ボタン（ハートアイコン）から、記事にいいねを付けられます。

```
legacy:
  image: some/amd64-only-image
  platform: linux/amd64
```

ただし、可能ならマルチアーキ対応の公式イメージを優先しましょう。

### よく使うコマンド

```
# 起動 (バックグラウンド)
docker compose up -d

# ログ追跡
docker compose logs -f api

# 特定のサービスだけ再ビルド
docker compose build api && docker compose up -d api

# DB に入る
docker compose exec db psql -U postgres -d app
```

### まとめ

- volumes でデータ消失を防ぐ
- platform 指定よりマルチアーキイメージを優先
- compose のコマンドはチームに共有する

 3

いいねをするとハートが赤くなり、いいね数が増えます。もう一度押すと取り消せます。

```
services:
  legacy:
    image: some/amd64-only-image
    platform: linux/amd64
```

ただし、可能ならマルチアーキ対応の公式イメージを優先しましょう。

## よく使うコマンド

```
# 起動 (バックグラウンド)
docker compose up -d

# ログ追跡
docker compose logs -f api

# 特定のサービスだけ再ビルド
docker compose build api && docker compose up -d api

# DB に入る
docker compose exec db psql -U postgres -d app
```

## まとめ

- volumes でデータ消失を防ぐ
- platform 指定よりマルチアーキイメージを優先
- compose のコマンドはチームに共有する



3

## 5-4. 記事にコメント・質問をする

記事詳細のさらに下には「リプライ」欄があります。「コメント」と「質問」を選んで投稿できます。

### リプライ (8)

コメント 質問

いい記事ですね！

8 / 2000

投稿する

すべて (3) コメント (2) 質問 (1)

質問 田中太郎 2026/04/25

M1 Mac だと一部のイメージで platform 警告が出るのですが、皆さんどう対処していますか？

← 回答する

▼ 返信 3 件を表示

コメント admin 2026/04/21

compose ファイルに env\_file を分けておくと、本番想定の実証も同じ手順で回せて便利でした。

▲ コメントする

リプライ欄は「すべて／コメント／質問」のタブで種類ごとに絞り込んで表示できます。



## 5-5. ベストアンサー

質問に対して付いた回答のうち、質問者は最も役立った回答を「ベストアンサー」に選ぶことができます。ベストアンサーは目立つ色で強調表示されます。



## 5-6. 他のユーザーのプロフィールを見る



記事の著者欄や、リプライ欄の投稿者名をクリックすると、そのユーザーのプロフィール画面に移動します。プロフィールでは、そのユーザーが「投稿した記事」を一覧で確認できます。



The screenshot shows the profile of a user named 山田花子 (Yamada Hanako). The profile includes a circular avatar of a dog, the name 山田花子, and a bio: Go言語とクリーンアーキテクチャに興味があります。 (I am interested in Go language and Clean Architecture). Below the profile, there are two tabs: 投稿した記事 (Articles I wrote) and いいねした記事 (Articles I liked). The 投稿した記事 tab is selected and underlined. It displays a list of articles. The first article is titled 'Vue 3 + Vuetifyで学ぶフロントエンド開発' (Learning Frontend Development with Vue 3 + Vuetify). It has tags for 'Vue' and 'フロントエンド' (Frontend), a date of 2026/04/15, and a heart icon with the number 3, indicating 3 likes. The page has a blue header with the KD-Te.ch logo and a 'ログアウト' (Logout) button.

タブを切り替えると、そのユーザーが「いいねした記事」も確認できます。



This screenshot shows the same user profile for 山田花子, but the 'いいねした記事' (Articles I liked) tab is now selected and underlined. The list of liked articles is displayed. The first article is 'Dockerで開発環境を統一する' (Unifying Development Environment with Docker), with tags for 'Docker' and '開発環境' (Development Environment), dated 2026/04/20, and has 3 likes. The second article is 'Goで作るREST API入門' (Introduction to REST API with Go), with tags for 'Go' and 'REST API', dated 2026/04/10, and also has 3 likes. The third article is '神戸大学でのハッカソン体験記' (Hacking Experience at Kobe University), with tags for 'ハッカソン' (Hackathon) and '神戸大学' (Kobe University). The page layout and header are consistent with the previous screenshot.

## 6. 評価してほしいポイント

### 6-1. 統一された UI / UX

UI フレームワークに **Vuetify** を採用し、色・サイズ・余白・コンポーネントのルールを揃えました。開発側の負荷を抑えながら、画面全体で統一感のある見た目・操作感を実現しています。

### 6-2. Markdown で気持ちよく書ける記事エディタ

記事の作成・編集は Markdown ベースで、書き味にこだわりました。

- 見出し・太字・リスト・コードブロックなどを挿入できる **マークダウンツールバー**
- **Ctrl+B (太字)** などのショートカット で素早く装飾を入力可能
- 箇条書きの途中で改行すると、自動で次の行も箇条書きが継続される
- コードブロックは、編集集中のプレビューと記事詳細の両方で **言語別の色付きハイライト** で表示

### 6-3. 読みやすい記事 UI

投稿された記事は、Markdown を整形して見やすく表示します。コード・見出し・引用などが崩れず読めるので、技術メモや解説記事との相性が良い構成になっています。

### 6-4. 多機能なプロフィールと「ユーザー同士の透明性」

- プロフィール画面で **自分のアイコン画像を変更** できる
- 自分の投稿記事だけでなく、**他人がいいねした記事も一覧で見られる**
- 興味のあること・活動内容など、ユーザー像が一目で分かる

「誰がどんな記事に反応しているか」が見えることで、コミュニティとしての透明性を高めました。

### 6-5. コメント／質問の絞り込み

リプライ欄は「コメント」と「質問」の2種類を投稿でき、**タブで種類ごとに絞り込み表示**が可能です。記事に対するフィードバックの種類を整理しやすくしています。

### 6-6. スクラムを意識した開発プロセス

- 最小限の構成を意識し、機能単位で **スクラム開発** を回した
- **設計についてチームで積極的に共有・議論** しながら進めた
- 他チームと比べても **読みやすい Pull Request** を出すことを意識し、レビューしやすい単位での開発に取り組んだ
- GitHub の **issue / PR 番号が 300 を超える** ほど、issue・PR ベースの開発を徹底した

### 6-7. バックエンドの実装方針

- ORM (O/R マッパー) を使わず、**生の SQL を書いて実装**
- API は OpenAPI (Swagger) を真のソースとし、TypeScript クライアントを自動生成する **コントラクト駆動** の構成

---

## 7. 苦労した点・工夫した点

## 7-1. Git / GitHub の運用に慣れるまで

開発初期はチームの誰もが GitHub の issue / PR 運用に慣れておらず、

- issue の立て方が雑で、何をすべきかが伝わりにくかった
- PR に動作確認の手順がなく、レビューがしづらかった
- コードレビューのやり方そのものが分からなかった
- Copilot のレビューコメントに対し、知識不足でどう対応すべきか判断しづらかった
- コンフリクトの解消にも何度も苦戦した

ここを乗り越えるために、issue / PR のテンプレートや動作確認の項目を整えながら、徐々に **見やすく追いやすい開発履歴** を残せるように改善していきました。

## 7-2. AI との付き合い方

開発の各所で AI（ChatGPT / Copilot 等）を活用しましたが、使いこなすこと自体が大きな課題でした。

- ハルシネーションを起こさせないために、**どこまで AI に任せるか** の線引きが難しい
- AI が出力した内容を理解しきれず、深掘りや議論ができない（質問力不足）
- チャット型 AI に **どこまでコンテキストを読み込ませるか** の判断
- 「ネット検索で調べる」「AI に聞く」「自分の知識として持つ」のバランスを取るのが難しい

使い込むうちに、AI に任せる範囲を意識的に区切り、出力を必ず自分で読み解いてから取り込むスタイルに変えていきました。

## 7-3. バックエンドの知識不足

チーム全体としてバックエンドの知識が薄く、

- Go / SQL を触ること自体に時間がかかった
- バックエンドで起きていることを言語化できず、**スクラム開発の「透明性」** を保つのが難しかった

設計の意図やデータの流れをチームで共有する場を意識的に増やし、専門用語を避けて説明する工夫を重ねました。

## 7-4. タスクの依存関係と優先順位

機能間で依存関係があり、思ったように並列で進められない場面が多くありました。何を先にやるべきか・どれを後回しにできるかをスクラムの中で確認しながら進める難しさを学びました。

## 7-5. 開発環境（Windows）まわり

- Windows 環境で動作が重かったり、拡張機能の導入が手間だったりした
- ホームディレクトリのユーザー名が日本語のため、一部のツールが導入できなかった
- 最終的に Linux 環境を導入するメンバーもあり、その移行自体が一苦労だった

## 7-6. チーム内の知識共有

メンバー間で知識量に差があり、共有のしかたに毎回工夫が必要でした。専門用語を噛み砕いて説明したり、設計について議論する時間を取ったりしながら、少しずつチーム全体の理解度を引き上げていきました。

## 7-7. 想定外: GitHub Education の Copilot プラン

GitHub Education の Copilot 課金プランを当初の前提に組み込んでいましたが、申請直前（4/22 ごろ）に新規登録での課金プラン利用ができなくなり、開発計画を組み直す必要が生じました。

## 7-8. 働きすぎによる生産性低下

休み時間にも作業を続けてしまい、長時間取り組んでいるわりに集中力が落ち、結果的に生産性が下がっていた時期がありました。「きちんと休むことも開発の一部」だと振り返ることができた点も、チーム制作を通じての学びです。

## 7-9. 公開して、誰でも触れるようにしたこと

作って終わりではなく、インターネット上に公開して **URL を開けば誰でも試せる** 状態にしました。無料のサービスで公開しているため、しばらく誰もアクセスしないと一時的に表示が遅くなることがあります（[4. デプロイ環境](#) に注意書きを記載）。安定して動かしたい場合は、手元のパソコンで動かす方法もご用意しています。